2025 21st IEEE International Conference on Mechatronic and Embedded Systems and Applications (MESA)
November 8-10, 2025. Macau, China

2025 21st IEEE International Conference on Mechatronic and Embedded Systems and Applications (MESA) | 979-8-3315-7553-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/MESA68091.2025.11278886

Bridging Sim2Real – Digital Twin for Autonomous Mobile Robots

Xin Yan Ding
Lee Kong Chian Faculty of Engineering and Science
Universiti Tunku Abdul Rahman
Bandar Sungai Long, 43000
Kajang Selangor, Malaysia
dingyanyan483@gmail.com

Wun-She Yap
Lee Kong Chian Faculty of Engineering and Science
Universiti Tunku Abdul Rahman
Bandar Sungai Long, 43000
Kajang Selangor, Malaysia
yapws@utar.edu.my

Fei-Yue Wang
State Key Laboratory for Managt and Control of Complex Systems
Institute of Automation
Chinese Academy of Sciences
Beijing 100190, China
feiyue.wang@ia.ac.cn

Wenwen Ding
DeSci CPI
University Research and Innovation Center
Óbuda University
Budapest H-1034, Hungary
savanna.wen@gmail.com

Jingwei Ge
DeSci CPI
University Research and Innovation Center
Óbuda University
Budapest H-1034, Hungary
gjw19@tsinghua.org.cn

Danny, Wee-Kiat Ng
Lee Kong Chian Faculty of Engineering and Science
Universiti Tunku Abdul Rahman
Bandar Sungai Long, 43000
Kajang Selangor, Malaysia
ngwk@utar.edu.my

Choon-Hian Goh
Lee Kong Chian Faculty of Engineering and Science
Universiti Tunku Abdul Rahman
Bandar Sungai Long, 43000
Kajang Selangor, Malaysia
gohch@utar.edu.my

Lee Cheun Hau*
Lee Kong Chian Faculty of Engineering and Science
Universiti Tunku Abdul Rahman
Bandar Sungai Long, 43000
Kajang Selangor, Malaysia
haulc@utar.edu.my

***Abstract*—As Autonomous Mobile Robots (AMRs) see growing adoption in logistics, manufacturing, and services, the need for reliable Digital Twin (DT) systems for real-time monitoring, control and performance optimization is increasing. However, the development of accessible DT systems remains challenging due to proprietary tool dependence and synchronization limitations between physical robots and their virtual counterparts. This study addresses these issues by leveraging open-source tools, specifically ROS2 and Isaac Sim, to develop a DT system prototype aimed at promoting wider DT adoption in AMR applications. A digital model of the environment was created and integrated into Isaac Sim, followed by the development of a functional DT system. Real-time data transfer between the physical robot and DT was validated through a series of tests. The system mirrored the AMR's behavior and data relatively well, enabling navigation control and data analysis through a custom user interface. Mirroring accuracy was quantified, showing positional and orientation errors within 0.3 meters and 10°, respectively. The DT also featured a path preview function, with travel time estimation errors below 3.5 seconds. Future work will enhance the fidelity of the digital environment, refine robot physics in simulation, and incorporate machine learning models for predictive maintenance.**

***Keywords*—Digital Twin, Autonomous Mobile Robots, ROS2, Isaac Sim, Sim2Real**

I. Introduction

AMRs are increasingly deployed in manufacturing, logistics, and healthcare for tasks such as material transport and delivery [1]. Their autonomy reduces manual labour, lowers operational costs, and enhances productivity [2]. While significant progress has been made in AMR mobility, navigation, and task execution [3], there remains a critical gap in realizing closed-loop, real-time DT frameworks that can seamlessly connect simulation and real-world operations. Existing DT studies focus on conceptual models or proprietary implementations, which hinder scalability and practical integration. For instance, Cakir et al. [4] proposed a YANG-based DT middleware with strong technical ground, yet its complexity and long development cycle limit adoption and real-time (RT) validation in operational environments. Conventional simulation tools such as Gazebo and CoppeliaSim are valuable for pre-deployment testing, bridging the initial Sim2Real gap by validating control algorithms and sensor integration before deployment [5]-[9]. However, their usefulness typically ends once the robot is deployed. They lack the ability to continuously synchronize with live sensor data or provide feedback to refine models during operation. As a result, developers must rely on repeated offline simulations to maintain model fidelity, which increases development time and cost. A real-time DT framework, in contrast, enables ongoing synchronization between physical and virtual systems, facilitating dynamic updates, predictive control, and system-wide optimization [10]-[12]. For instance, if an AMR encounters an obstacle, the DT can instantly adjust fleet-wide path planning [13].

Recent research indicates that AMR DT are evolving from validated simulation environments into integral components for complex autonomous systems. Priority has been put in advancing from RT DT toward interactive and cognitive DT [14], integrating deep reinforcement learning agents, e.g., PPO and SAC with human-robot interfaces for informed, real-time decision-making [15]. Besides, scaling simulation infrastructure using distributed architectures like TeraAgent [16] is crucial for systemic governance across large multi-robot fleets. Moreover, standardized frameworks for sensor fusion and fidelity modeling are also important to ensure accurate localization in dynamic environments, leveraging high-resolution physics kernels such as AMR-transformer methodologies [17]. To counter high implementation costs, the principal barrier to adoption of DT, investments should be strategically directed toward high-value, complex domains, such as adaptive manufacturing and mobile manipulation systems, where DT demonstrably reduce deployment risks

and accelerate operational readiness [18]. Thus, to bridge this gap, this paper introduces a real-time, closed-loop DT system for AMRs, built entirely on open-source tools. ROS2 [19] is utilized as a data bridge between the physical and virtual environments, while NVIDIA Isaac Sim [20] provides a high-fidelity simulation platform. The DT continuously mirrors the live state of the AMR such as location, task progress, and battery condition via ROS2 topics, allowing operators to visualize, test, and validate actions in simulation before applying them in the real world. This closed-loop feedback mechanism transforms traditional Sim2Real testing into a Sim-with-Real paradigm, where both domains evolve together during operation.

This development is part of an ongoing project at Universiti Tunku Abdul Rahman (UTAR) involving the deployment of AMRs with automated battery hot-swapping capabilities. These AMRs are designed to perform uninterrupted "touch-and-go" operations by autonomously replacing depleted battery packs instead of waiting for recharge cycles. The proposed DT system provides the technological backbone for this application, enabling integrated fleet monitoring, battery-state visualization, and real-time coordination between robots and the battery-swapping stations. Beyond tool integration, this paper showcases how the DT facilitates closed-loop verification, from simulation to real-time operation, and extends to practical, energy-centric applications such as automated battery hot-swapping. This paper focuses on the design and validation of the DT system. Future publications will cover: (1) the IoT communication framework establishing robust data pipelines between physical assets and the DT, and (2) the battery hot-swapping system, which aims to minimize halt time and maximize operational efficiency of AMR.

II. Proposed DT System

Fig. 1 illustrates the proposed DT and Internet-of-Things (IoT) system framework, which integrates both the virtual and physical environments of the AMR and the automatic battery swapping station via an MQTT server and a Virtual Private Network (VPN). The DT is hosted on a local PC running ROS2, with the virtual environment developed using NVIDIA Isaac Sim. The same PC is also configured with Node-RED, InfluxDB, and Grafana to store and publish the AMR's data to the cloud. The system requirements for NVIDIA Isaac Sim and the PC specifications are listed in TABLE I.

TABLE I. System Requirements for Isaac Sim [20]

Element	System Requirements for NVIDIA Isaac Sim			Specification of the PC
	Min	Good	Ideal	
OS	Ubuntu 20.04/22.04 Windows 10/11			Ubuntu 22.04 LTS
CPU	Intel i7 Ryzen 5	Intel i7 Ryzen 7	Intel i9 Ryzen 9	Ryzen 9 7900x
Cores	4	8	16	12
RAM	32GB	64GB	64GB	64GB
GPU	GeForce RTX 3070	GeForce RTX 4080	RTX Ada 6000	GeForce RTX 4070 Super
VRAM	8GB	16GB	48GB	12GB

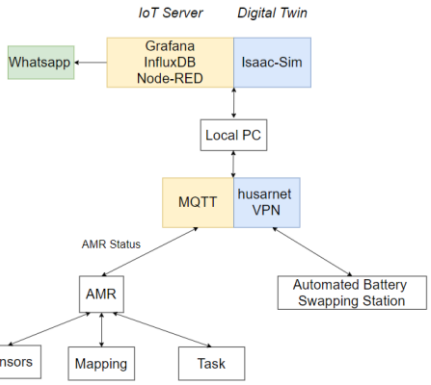
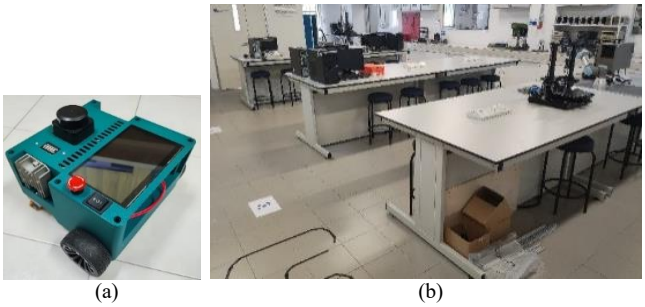


Fig. 1. Block Diagram of the DT and IoT System with AMR.



(a) (b)

Fig. 2. Photo of the (a) Mini AMR and (b) Test Environment.

Real-time data from the AMR's sensors, battery modules, and other components are published and subscribed through the MQTT server, enabling both the DT system and Grafana dashboards to reflect the most up-to-date information for local and cloud-based visualization. Control commands can also be transmitted to the AMR through the same channel. Briefly, an automatic battery swapping station is also integrated into the larger system to replace the AMR's battery modules, removing the need for prolonged docking at charging stations. Data from the swapping station is transmitted through the same communication infrastructure, ensuring seamless integration and operational efficiency.

This section outlines the approach and development of a DT system prototype for an AMR. Isaac Sim is used to host the DT on a remote PC, which was connected to the robot's onboard computer for data exchange. As shown in Fig. 2, the robot developed by Danny Wee-Kiat *et al.* [22], along with a selected environment from the university, was modeled and displayed within Isaac Sim. A custom user interface (UI), discussed in Section II.B was also developed to display key real-time status information and provide control capabilities for the physical robot via the DT environment.

A. Digital Twin Setup

The development of the digital model begins by converting the robot design from Fusion 360 into a Unified Robot Description Format (URDF) file to ensure compatibility with Isaac Sim. Following this, the robot's operating environment is digitally reconstructed using photogrammetry to create a realistic and dimensionally accurate virtual space.

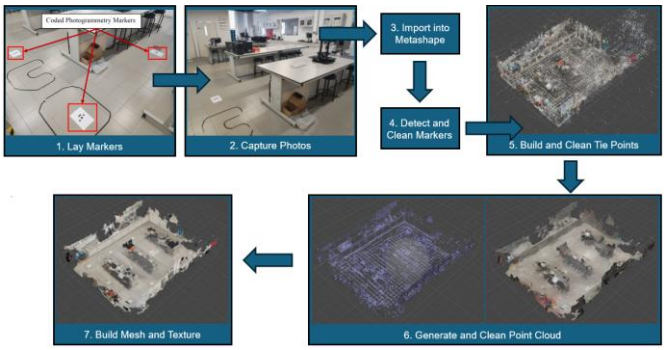


Fig. 3. Flowchart of Photogrammetry Model Cosntruction.

Prior to image capture, photogrammetry markers were strategically placed throughout the test environment, and inter-marker distances were measured to enhance modeling precision. A total of 1,198 images were captured with a 64MP Poco F4 smartphone along a circular route around the laboratory and processed in Agisoft Metashape, as illustrated in Fig. 3. The software's automated marker detection was refined manually to remove false detections, and scale bars were introduced to ensure real-world scaling. Photos were aligned to estimate camera positions, followed by noise filtering and iterative camera optimization. Tie points were refined using three key parameters namely reconstruction uncertainty (set to 10), projection accuracy (level 5), and reprojection error (iteratively minimized) to ensure geometric accuracy and model rigor.

After filtering, a clean point cloud and mesh were generated, and realistic textures were applied from the captured images. The resulting textured mesh was then exported as an *.obj* file and imported into Isaac Sim, providing a photorealistic virtual test environment aligned with real-world features such as charging stations and obstacles. This setup enables operators to simulate AMR workflows prior to physical deployment within Isaac Sim, the DT interface integrates robot control, battery monitoring, and path planning. A custom UI extension is being developed using Omniverse's Python API to display real-time data, including battery voltage, state of charge, and robot coordinates. Operators can execute pre-defined routes validated through DT simulations or set custom waypoints with estimated travel time and distance. The system also integrates ROS 2 topics, enabling seamless bidirectional communication between the physical and virtual systems. Additionally, the SLAM Toolbox in ROS 2 was used to generate a 2D map of the test environment to support the robot's navigation function, as shown in Fig. 4.

B. Digital Twin User Interface

The DT UI was designed to display the robot's real-time position within the environment while also providing access to key information such as battery status and basic control functions. NVIDIA Omniverse's built-in UI Extension framework was utilized to implement this interface. The UI was developed in Isaac Sim using the built-in UI Extensions feature, with Python scripts integrating ROS2 elements to enable real-time communication between ROS2 nodes and the UI. Four core functions were implemented:

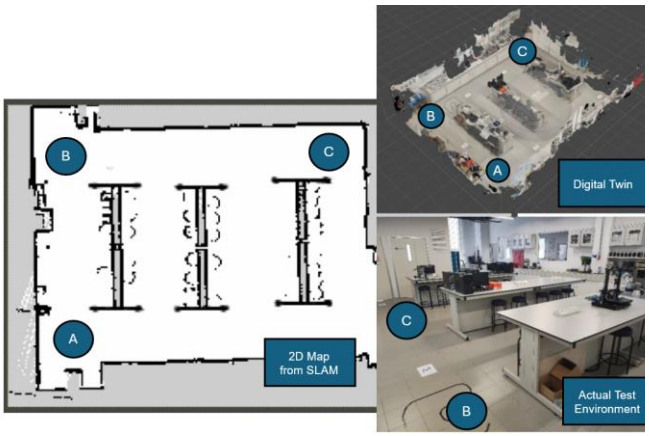


Fig. 4. SLAM Generated Map of the Test Environment and its Digital Twin and Actual Counterparts.

(1) *Battery Status Display:* A dedicated ROS2 node was developed to subscribe to a ROS2 topic transmitting battery data from the robot. This information, including voltage and state of charge (SOC) of battery, is displayed on the UI whenever data is received.

(2) *Robot Status Display:* The UI presents the robot's real-time position as numerical coordinates and a graphical grid-based representation for easier interpretation. A ROS2 node retrieves positional data, which Isaac Sim uses to update the virtual robot's location, mirroring the physical robot.

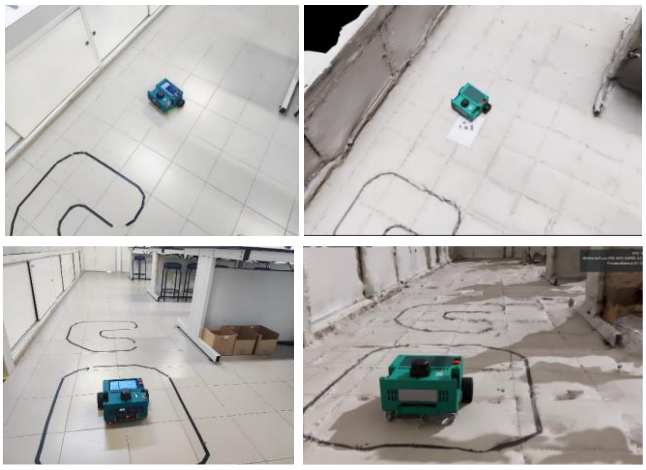
(3) *Robot Controls:* Control functions are embedded within the UI, with ROS2 nodes enabling communication with the robot's navigation system. Available controls include a "Home" button, custom waypoint navigation, and execution of preset routes. A cursor object in Isaac Sim allows users to select waypoints, which are then transmitted to the robot for execution.

(4) *Path Preview:* A basic data analysis feature calculates the Estimated Travel Time (ETT) and distance to the goal after custom waypoints are selected. Upon pressing the "Preview" button, the script uses ROS2's Nav2 server function ComputePathThroughPoses to retrieve a list of waypoints along the path. The total distance is calculated by summing the lengths between each waypoint, and the ETT is derived by dividing this distance by the robot's average velocity.

III. Validation of Isaac Sim DT

A. Built Digital Model

The final prototype of the DT demonstrated satisfactory functionality. The virtual robot model accurately mirrored the movements and behavior of its physical counterpart, as illustrated in Fig. 5. A UI was successfully developed to display essential robot status information, including real-time location and battery level, as shown in Fig. 6. The UI also included an interactive control panel, implemented using ROS2 and Python, which enabled users to navigate the robot, estimate the distance to the goal, and calculate the ETT directly from the DT system.



(a) (b)

Fig. 5. Side-by-side View of the (a) Real Robot and (b) its Virtual Model in Isaac Sim.

The environment model was successfully generated using Agisoft Metashape. The resulting virtual environment closely resembled the physical setting, with high visual fidelity that made it possible to identify distinct features and corners of space, as shown in Fig. 7. Although the digital model was not perfect, the minor imperfections were primarily due to reflective surfaces and inconsistent lighting, factors that were difficult to control, particularly given that most physical assets in the environment were light-colored or white.

B. User Interface Functionality

Fig. 6 displays the final UI, which successfully achieved all intended functionalities while providing a clear and organized overview of the system's status. Key features like the Battery Status, Robot Location, and Robot Controls were grouped into distinct frames on the right for better clarity and usability. All UI buttons were tested and performed reliably and executed their respective tasks as intended. For example, when the *Home* button was pressed, the robot successfully navigated from its current location back to the designated home point (identified as location 'B' in Fig. 4). This confirmed that the home position was correctly transmitted over the network and properly executed by the robot.

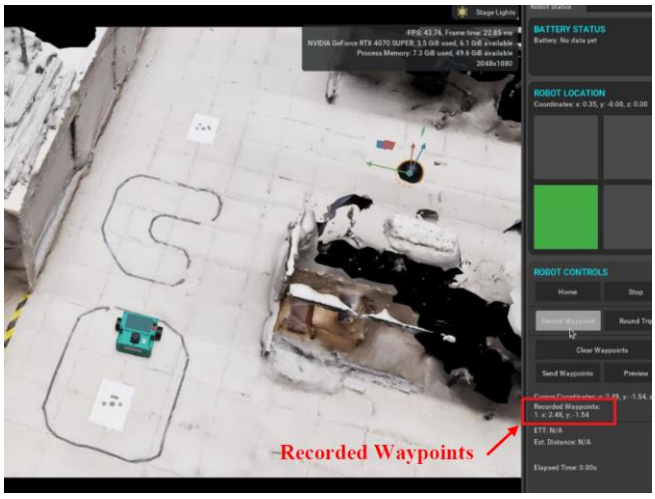


Fig. 6. Isaac Sim DT with Custom UI.

(a) (b)

Fig. 7. Side-by-side View of the (a) Real Environment and (b) Virtual Environment in Isaac Sim.

Similarly, the manual waypoint selection functioned as intended. Users could move the waypoint cursor to a desired location and press the *Record Waypoint* button, as illustrated in Fig. 6. Pressing the *Send Waypoints* button then successfully transmitted the recorded waypoints to the robot for execution. The preset route function also performed effectively. Pressing the *Round Trip* button-initiated navigation along a predefined set of waypoints, confirming proper communication and execution within the DT control framework.

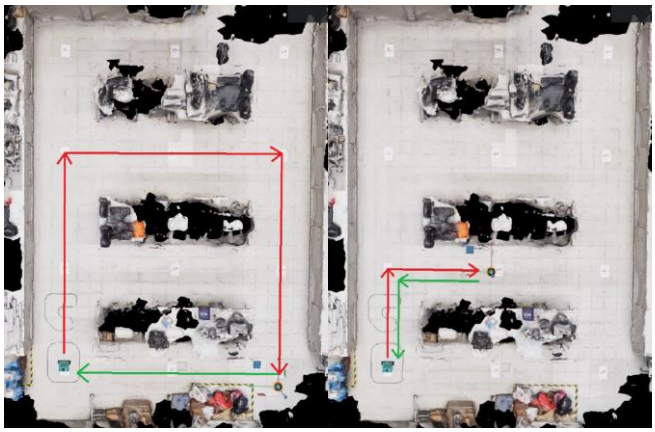
IV. Evaluation of the Proposed DT

A. Robot Pose Twinning Accuracy

Two tests were conducted to evaluate the accuracy of the robot's pose mirroring in the DT system. The paths for the two test cases namely *Circle Trip* and *Round Trip* are shown in Fig. 8. The robot's pose comprises its position (X and Y coordinates) and orientation (yaw angle). Accuracy was measured at the goal point for each cycle, which consisted of the robot traveling from the home location to the goal and returning home.

An error margin of up to 0.3 meters in both X and Y coordinates and up to 10° in yaw angle was defined as these values strike a balance between realistic physical performance and DT synchronization fidelity. In practice, minor discrepancies between the physical and virtual robot poses can arise from factors such as wheel slippage, sensor noise, latency in data transmission, and environmental variations. Through repeated trials, it was observed that deviations within 0.3 m in position and 10° in orientation did not significantly affect the robot's ability to complete its navigation tasks accurately or compromise the effectiveness of real-time mirroring in the DT environment. Therefore, these thresholds were defined to ensure that the DT reflects the robot's real-world state with sufficient accuracy for control validation and closed-loop testing, while accommodating the inherent uncertainties of real-world operation.

(1) *Circle Trip Test:* In this test, the robot was instructed to navigate a circular path through the environment, reaching a designated goal and then returning home, as illustrated in Fig. 8(a). Based on the results in Fig. 9, positional errors decreased, while yaw angle errors increased. The reduction in positional error may be attributed to improved SLAM localization due to multiple turns, while the increased number of turns likely contributed to higher yaw deviations.



(a) (b)

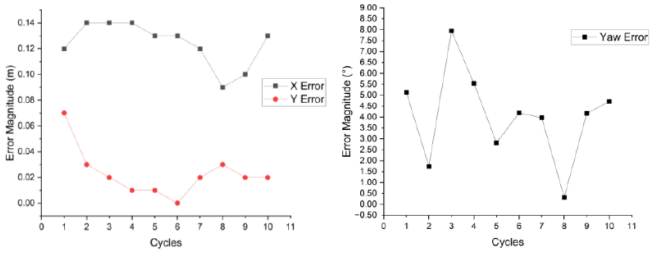
Fig. 8. Path Overview for the (a) Circle Trip and (b) Round Trip Test.

(2) *Round Trip Test:* For this test, the robot was required to perform a 90° turn enroute from home to the goal and back, as shown in Fig. 8(b). Results in Fig. 10 show that positional errors in both X and Y coordinates were consistently low, with final errors at the goal and home points under 0.08 meters. The yaw angle showed a trend of increasing deviation upon reaching the goal, followed by a sharp correction in later cycles. This suggests that the localization algorithm gradually refined the robot's maneuvering accuracy. Importantly, yaw angle errors remained within the 10° threshold throughout all test cycles.

B. Estimated Travel Time Accuracy

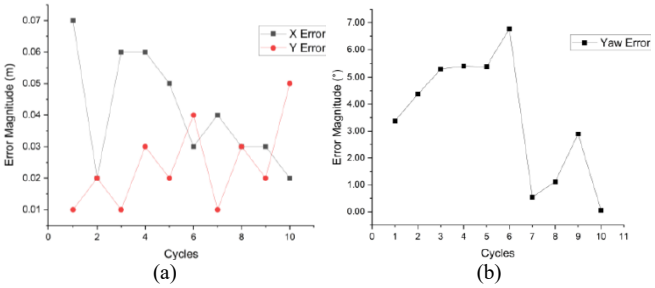
During the tests described in the previous subsection, the ETT feature was also evaluated by comparing the pre-trip estimates with the actual travel times. Before each trip, the ETT was previewed, and the elapsed time from departure to goal arrival was recorded. The differences between the estimated and actual travel times were plotted, as shown in Fig. 11.

The number of overestimations and underestimations was found to be evenly distributed, with a similar count for each. Positive deviations (i.e., estimated time greater than actual time) suggest that the robot dynamically discovered a shorter path than initially planned. Conversely, negative deviations indicate that the planned path could not be fully executed, which may be attributed to factors such as wheel slippage requiring path correction or dynamic obstacle avoidance that extended the travel duration.



(a) (b)

Fig. 9. Circle Trip Test: Graphs of (a) Positional and (b) Yaw Error Over Test Cycles.



(a) (b)

Fig. 10. Round Trip Test: Graphs of (a) Positional and (b) Yaw Error Over Test Cycles.

Across 20 measurements, the maximum overestimation error ranged from 0 to 3 seconds, while the maximum underestimation error ranged from -5 to 0 seconds. Overall, the ETT estimation feature demonstrated satisfactory performance as a basic data analysis tool, with potential for further enhancement through more advanced prediction models or adaptive algorithms.

C. Latency Test

Since a DT system is expected to operate in real time (with ideally zero latency), a latency test was conducted to evaluate its responsiveness. This test was performed alongside the previously mentioned experiments, specifically by measuring the time taken for the robot to respond after receiving a navigation command. At the moment the *Send Waypoints* button was pressed to transmit the recorded waypoints, a stopwatch was started. The stopwatch was stopped immediately once the robot began to move, and the elapsed time was recorded. A total of 20 latency measurements were collected.

Fig. 12 presents a plot of the recorded latency values. It is important to note that, as this was a manually operated test using a stopwatch, slight delays were introduced due to human reaction time when stopping the timer. As such, the recorded latencies may slightly overestimate the actual system response times. A few higher-latency outliers are visible in the plot, likely caused by intermittent instability in the network connection between the robot and the Isaac Sim DT. This is attributed to the use of a mobile hotspot, which provides lower bandwidth and reliability compared to a standard Wi-Fi router. Despite this, most measurements fall below 2 seconds, with majority under 1 second.

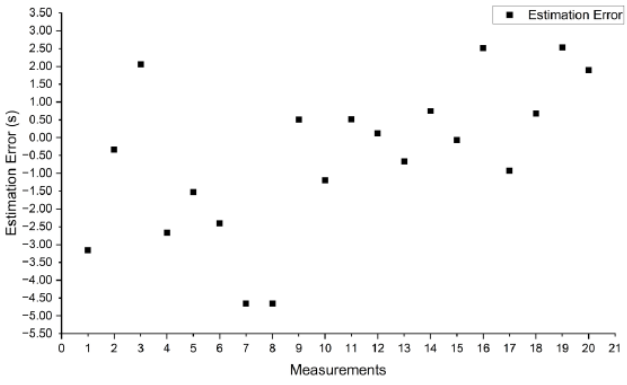


Fig. 11. Graph of Travel Time Estimation Error Over Measurements.

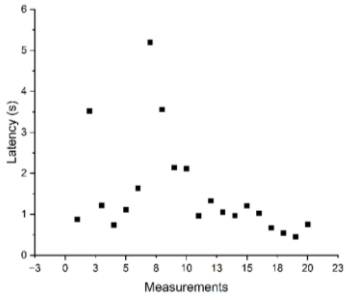


Fig. 12. Graph of Latency Over Measurements.

V. CONCLUSIONS

This study successfully developed a DT prototype for AMRs using open-source tools like ROS2 and Isaac Sim to enable RT status monitoring and control within a geometrically accurate virtual environment. Synchronization between the physical and virtual AMR was achieved with pose errors maintained within 0.3 m for position and 10° for orientation, while the ETT prediction-maintained errors below 3.5 seconds. By leveraging open-source frameworks, this work addresses the lack of accessible and cost-effective DT solutions, laying a foundation for scalable and adaptable DT systems that support efficient lifecycle management and continuous operational optimization. While the prototype validates the DT concept and demonstrates practical feasibility, future work should enhance environmental realism, physics calibration, and intelligent automation. Improvements may include higher-quality photogrammetry, larger image datasets, or CAD-based modeling with realistic textures to increase visual fidelity. Expanding the framework to support coordinated multi-AMR operations in complex industrial settings will be essential to bridge the gap between research and real-world deployment, advancing toward fully data-driven and autonomous AMR ecosystems.

ACKNOWLEDGMENT

This work was supported by the Universiti Tunku Abdul Rahman, Lee Kong Chian Faculty of Engineering and Science, and Department of Mechatronics and Biomedical Engineering in partnership with AIR.

REFERENCES

[1] R. Hercik, R. Byrtus, R. Jaros, and J. Koziorek, "Implementation of autonomous mobile robot in SmartFactory," *Applied Sciences*, vol. 12, no. 17, p. 8912, Sep. 2022, doi: 10.3390/app12178912.

[2] M. Boccia, A. Masone, C. Sterle, and T. Murino, "The parallel AGV scheduling problem with battery constraints: A new formulation and a matheuristic approach," *European Journal of Operational Research*, vol. 307, no. 2, pp. 590–603, Oct. 2022, doi: 10.1016/j.ejor.2022.10.023.

[3] A. Loganathan and N. S. Ahmad, "A systematic review on recent advances in autonomous mobile robot navigation," *Engineering Science and Technology an International Journal*, vol. 40, p. 101343, Feb. 2023, doi: 10.1016/j.jestch.2023.101343.

[4] L. V. Cakir, T. Bilen, M. Özdem, and B. Canberk, "Digital Twin Middleware for Smart Farm IoT Networks," *2023 International Balkan Conference on Communications and Networking (BalkanCom)*, Jun. 2023, doi: 10.1109/balkancom58402.2023.10167962.

[5] A. D. Haridevan, J. Kang, M. Yuan, and J. Shan, "ROS2-Gazebo Simulator for drone applications," *2022 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 1232–1238, Jun. 2024, doi: 10.1109/icuas60882.2024.10556903.

[6] J. Platt and K. Ricks, "Comparative analysis of ROS-Unity3D and ROS-Gazebo for mobile ground robot simulation," *Journal of Intelligent & Robotic Systems*, vol. 106, no. 4, Dec. 2022, doi: 10.1007/s10846-022-01766-2.

[7] B. Bogaerts, S. Sels, S. Vanlanduit, and R. Penne, "Connecting the CoppeliaSim robotics simulator to virtual reality," *SoftwareX*, vol. 11, p. 100426, Jan. 2020, doi: 10.1016/j.softx.2020.100426.

[8] A. Farley, J. Wang, and J. A. Marshall, "How to pick a mobile robot simulator: A quantitative comparison of CoppeliaSim, Gazebo, MORSE and Webots with a focus on accuracy of motion," *Simulation Modelling Practice and Theory*, vol. 120, p. 102629, Jul. 2022, doi: 10.1016/j.simpat.2022.102629.

[9] G. Montenegro *et al.*, "Modeling and control of a spherical robot in the CoppeliaSim simulator," *Sensors*, vol. 22, no. 16, p. 6020, Aug. 2022, doi: 10.3390/s22166020.

[10] H. Park *et al.*, "Integration of an exoskeleton robotic system into a digital twin for industrial manufacturing applications," *Robotics and Computer-Integrated Manufacturing*, vol. 89, p. 102746, Mar. 2024, doi: 10.1016/j.rcim.2024.102746.

[11] P. Stączek, J. Pizoń, W. Danilczuk, and A. Gola, "A Digital Twin Approach for the Improvement of an Autonomous Mobile Robots (AMR's) Operating Environment—A Case study," *Sensors*, vol. 21, no. 23, p. 7830, Nov. 2021, doi: 10.3390/s21237830.

[12] S. Wang, J. Zhang, P. Wang, J. Law, R. Calinescu, and L. Mihaylova, "A deep learning-enhanced Digital Twin framework for improving safety and reliability in human–robot collaborative manufacturing," *Robotics and Computer-Integrated Manufacturing*, vol. 85, p. 102608, Jul. 2023, doi: 10.1016/j.rcim.2023.102608.

[13] M. Denk, S. Bickel, P. Steck, S. Götz, H. Völkl, and S. Wartzack, "Generating digital twins for Path-Planning of autonomous robots and drones using constrained homotopic shrinking for 2D and 3D environment modeling," *Applied Sciences*, vol. 13, no. 1, p. 105, Dec. 2022, doi: 10.3390/app13010105.

[14] D. Mitchell *et al.*, "Cyber-physical-human systems for mobile robots in energy asset management: Current practices and future opportunities," *Energy and AI*, p. 100570, Aug. 2025, doi: 10.1016/j.egyai.2025.100570.

[15] P. Stączek, J. Pizoń, W. Danilczuk, and A. Gola, "A Digital Twin Approach for the Improvement of an Autonomous Mobile Robots (AMR's) Operating Environment—A Case study," *Sensors*, vol. 21, no. 23, p. 7830, Nov. 2021, doi: 10.3390/s21237830.

[16] L. J. Breitwieser, "Design and analysis of an Extreme-Scale, High-Performance, and modular Agent-Based simulation Platform," *arXiv.org*, Mar. 2025, doi: 10.3929/ethz-b-000726143.

[17] Z. Xu, J. Liu, K. Chen, Y. Chen, Z. Hu, and B. Ni, "AMR-Transformer: enabling efficient long-range interaction for complex neural fluid simulation," *arXiv.org*, Mar. 13, 2025. https://arxiv.org/html/2503.10257v1

[18] L. Bergs *et al.*, "Evaluating mobile robot navigation behavior in flexible assembly systems through digital twin and real-world experiments," *Springer Nature*, vol. 1, no. 1, Sep. 2025, doi: 10.1007/s44430-025-00010-4.

[19] A. Bonci, F. Gaudeni, M. C. Giannini, and S. Longhi, "Robot Operating System 2 (ROS2)-Based Frameworks for Increasing Robot Autonomy: A survey," *Applied Sciences*, vol. 13, no. 23, p. 12796, Nov. 2023, doi: 10.3390/app132312796.

[20] "Isaac SIM Requirements — Isaac SIM Documentation." https://docs.isaacsim.omniverse.nvidia.com/latest/installation/requirements.html

[21] N. D. Wee-Kiat, K. Ban-Hoe, C. S. Cheng, and C.-H. Goh, "3D Printed Differential Drive Robot with LiDAR and 3D Depth Camera," *2024 9th International Conference on Control and Robotics Engineering (ICCRE)*, pp. 1–5, May 2024, doi: 10.1109/iccre61448.2024.10589855.